1. Numeric differentiation
f(x) ,

$$\nabla_{\underline{x}} f(\underline{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

n forward 1

$$\frac{\partial f}{\partial x_i} = \frac{f(\underline{x} + \varepsilon \underline{e}_i) - f(\underline{x})}{\varepsilon}$$

$\varepsilon = 1 \times 10^{-8}$
1e-8

i^{th} basis vector $\longrightarrow \underline{e}_i = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \leftarrow i^{th} \text{ dimension} \\ \vdots \\ 0 \\ 0 \end{bmatrix}$

$$\hat{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \hat{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \hat{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

central deriv

Adv: easy
Disadv.: expensive.

$$\frac{\partial f}{\partial x_i} = \frac{f(\underline{x} + \frac{\varepsilon}{2} \underline{e}_i) - f(\underline{x} - \frac{\varepsilon}{2} \underline{e}_i)}{\varepsilon}$$

$O((n+1) \times$ comp. cost of eval $f)$

---

② Symbolic derivative (sympy)

f(x) = Symbols('x') + Symbol('y')

↓

formula for computing deriv → C code
→ simplify it
→ Latex math expr

Adv: faster than numerical deriv

Disadv: Not optimized for neural networks

Numerical deriv. : need data

$f(x)$ written as code in C
in Python
in Matlab

Sympy

$f(x)$ written as code in sympy

- sin
- cos
- exp
- log

} functions

- if conditions
- for loops

Automatic differentiation → Forward mode

→ Reverse mode (for NN)

pytorch
sin, +
cos, -
exp, @
log

$f(x)$ written as code in Pytorch

Do not mix libraries in implementing neural networks.

Diff between Forward mode and Reverse mode differentiation

$$f(g(h(x)))$$

FM $\rightarrow$ $\left(\frac{\partial f}{\partial g}\left(\frac{\partial g}{\partial h}\cdot\frac{\partial h}{\partial x}\right)\right)$

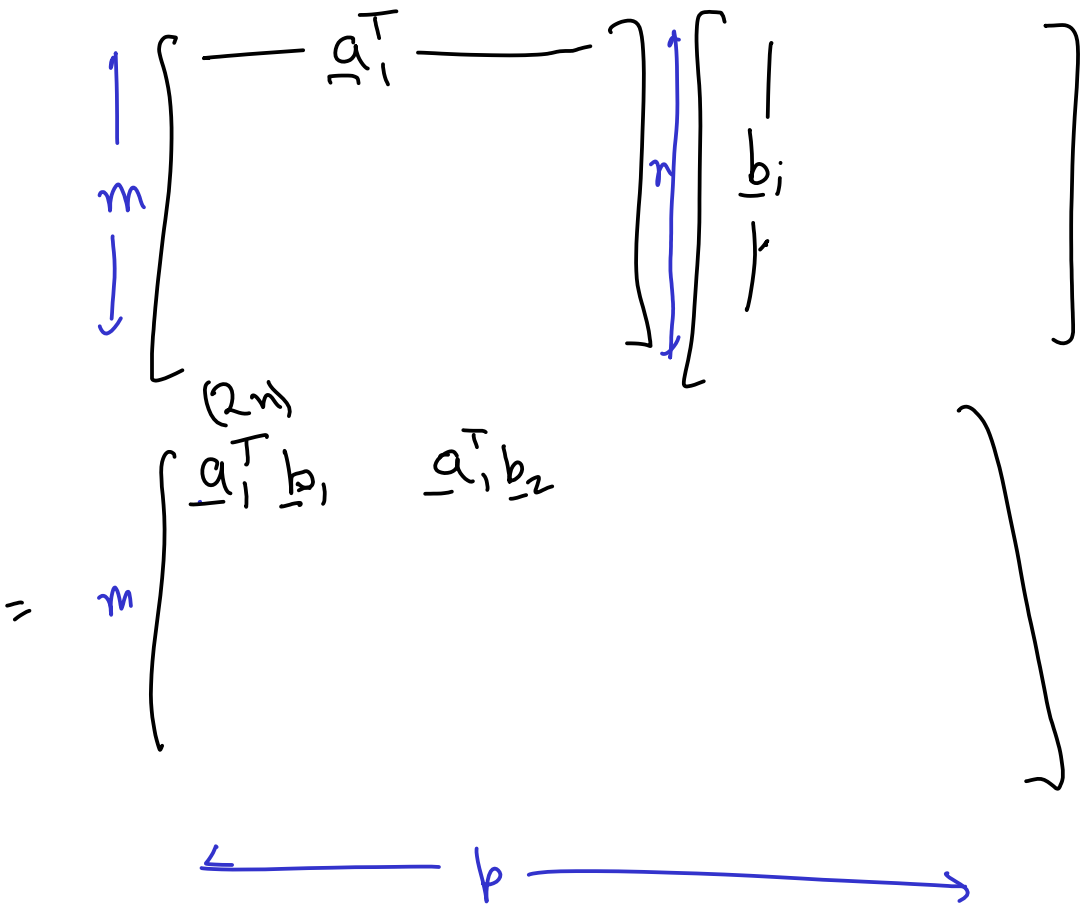
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g}\cdot\frac{\partial g}{\partial h}\cdot\frac{\partial h}{\partial x}$$

Jacobian matrices

RM $\rightarrow$ $\left(\left(\frac{\partial f}{\partial g}\cdot\frac{\partial g}{\partial h}\right)\cdot\frac{\partial h}{\partial x}\right)$

Computation cost

$C_{m\times p} = A_{m\times n}\cdot B_{n\times p}$ ← what is the cost? How many additions and multiplications?



$$\underline{x}^T\underline{y} = \sum_{i=1}^{n} x_i y_i$$

$n\times n$, $n\times n$

$O(n^3)$

$\geq O(n^{2.7})$

Cost = $2mnp$ operations

Reverse mode

$A_{m\times n}\ B_{n\times p}\ C_{p\times q}\ D_{q\times r}$

$m\times q$

$((A\,B)\,C)\,D_{q\times r}$ — $(A\,B)$: $m\times p$; $(\cdot\,C)$: $p\times q$

$2mnp + 2mpq + 2mqr$

$= 2m(np + pq + qr)$

$A(B(C\,D))$ ← Forward Mode

$2mnr + 2npr + 2pqr$

$(mn + np + pq)2r$

Neural Network

Loss output is always 1D

$\hat{y} = f_\theta(x)$

$\ell(y, \hat{y}) \in \mathbb{R}$ is always a scalar

$\ell(\hat{f}(g(h(x;\theta))))$

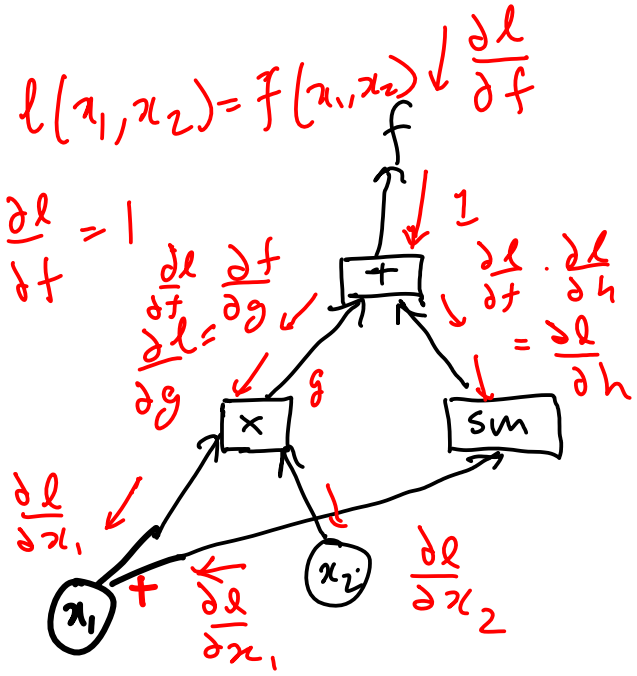
$$\frac{\partial \ell}{\partial \theta} = \left(\left(\left(\frac{\partial \ell}{\partial \hat{f}} \cdot \frac{\partial \hat{f}}{\partial g}\right)\frac{\partial g}{\partial h}\right)\frac{\partial h}{\partial \theta}\right)$$

$\frac{\partial \ell}{\partial \hat{f}}$: $1\times m$

IF the output dimensions > input dimensions then use forward mode instead of reverse mode and vice versa.

Computational Graphs

$$f(x_1, x_2) = x_1 \cdot x_2 + \sin(x_1)$$



Reverse mode differentiation (eventual output is 1-D: a loss function)

Directional Acyclic Graph (DAG)

$g(x_1, x_2) = x_1 x_2$

$h(x_1) = \sin(x_1)$

Back propagation is computing derivatives in neural networks using RM auto. diff.

$$l(x_1, x_2) = f(x_1, x_2) = g + h$$

$$\frac{\partial l}{\partial g} = \frac{\partial l}{\partial f} \frac{\partial f}{\partial g}$$

$$\frac{\partial l}{\partial g} = \frac{\partial l}{\partial f} \cdot 1, \quad \frac{\partial l}{\partial h} = \frac{\partial l}{\partial f} \cdot h$$

$$\frac{\partial l_g}{\partial x_1} = \frac{\partial l}{\partial g} \cdot \frac{\partial g}{\partial x_1} = \frac{\partial l}{\partial g} \cdot x_2$$

$$g(x_1, x_2) = x_1 \cdot x_2$$

$$\frac{\partial g}{\partial x_1} = x_2, \quad \frac{\partial g}{\partial x_2} = x_1$$

$$\frac{\partial l}{\partial x_2} = \frac{\partial l}{\partial g} \frac{\partial g}{\partial x_2} = \frac{\partial l}{\partial g} \cdot x_1$$

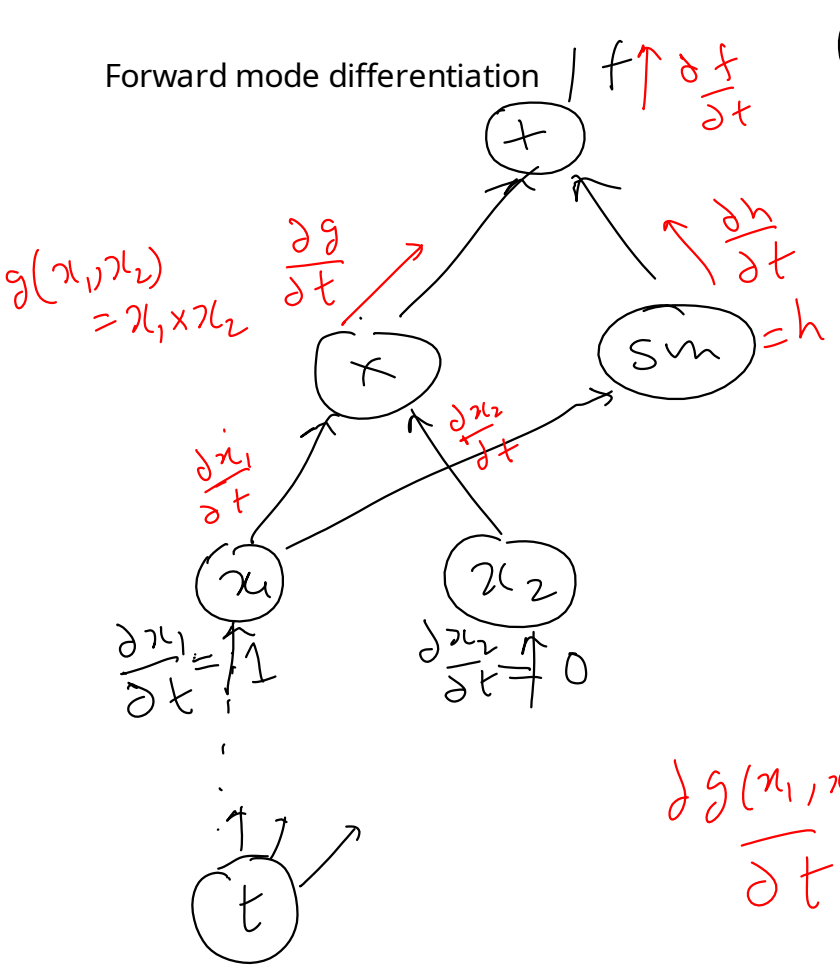
$$h(x_1) = \sin(x_1)$$

$$\frac{\partial l_h}{\partial x_1} = \frac{\partial l}{\partial h} \cdot \frac{\partial h}{\partial x_1} = \frac{\partial l}{\partial h} \cdot \cos(x_1)$$

$$\frac{\partial \ell}{\partial x_1} = \frac{\partial \ell_g}{\partial x_1} + \frac{\partial \ell_h}{\partial x_1} = \frac{\partial \ell}{\partial g} \cdot x_2 + \frac{\partial \ell}{\partial h} \cdot \cos(x_1)$$

We just backpropagated derivatives through a computational graph

Forward mode differentiation



RM 1-D output

FM 1-D input

$$\frac{\partial g(x_1, x_2)}{\partial t} = \frac{\partial g}{\partial x_1}\cdot\frac{\partial x_1}{\partial t} + \frac{\partial g}{\partial x_2}\frac{\partial x_2}{\partial t}$$

$$= x_2 \cdot \frac{\partial x_1}{\partial t} + x_1 \frac{\partial x_2}{\partial t}$$